Work Integrated Learning Programmes
M.Tech Software Engineering



Introduction to DevOps

Assignment 1

Implementing Automated CI/CD Pipelines for ACEest Fitness & Gym

By:

Name: Neel Gujar

BITS ID: 2025tm93111

Course: Introduction to DevOps (CSIZG514/SEZG514)

Assignment: Assignment 1

GitHub Repository:

https://github.com/Neel2211/ACEest-Fitness-Gym

Contents

1. Executive Summary 4
2. Project Objective 4
3. Application Overview 4
Features Implemented 4
Screenshot 1: Flask Application Running 5
Screenshot 2: Application Response 5
4. Version Control Strategy 6
Version Evolution Table 6
Feature Branches 6
Screenshot 3: Branches 7
Screenshot 4: Commit History 7
5. Unit Testing using Pytest 8
Test Coverage 8
Screenshot 5: Pytest 8
Test Execution Flow 9
6. Docker Containerization 9
Screenshot 6: Docker Build 9
Screenshot 7: Docker Run 10
Screenshot 8: Application response from container 10
7. GitHub Actions CI/CD Pipeline 11
CI/CD Flow Diagram 11
CI/CD Stages Executed 12
Screenshot 9: GitHub Actions Overview 12
Screenshot 10: Workflow details 15
8. Jenkins Build & Quality Gate 15
Jenkins Workflow 15
Screenshot 11: Jenkins Pipeline Configuration 17
Screenshot 12: Automatic Jenkins Trigger: Build started by SCM change 18

Screenshot 13: Environment Validation 18
Screenshot 14: Dependency Installation 19
Screenshot 15: Pytest Execution 19
Screenshot 16: Build Success 20
9. Release Management, Rollback and Hotfix Strategy 21
Release Validation Workflow 21
Screenshot 17: Incremental Version Builds 23
Screenshot 18: Release Failure 23
Screenshot 19: Rollback Success 24
Screenshot 20: Hotfix Success 25
10. Repository Structure 26
Screenshot 21: Repository Structure 27
GitHub Repository: https://github.com/Neel2211/ACEest-Fitness-Gym 27
11. Project Documentation (README.md) 27
README Contents 27
Screenshot 22: README Overview 28
Screenshot 23: README CI/CD Workflow 29
Screenshot 24: README Setup Instructions 29
12. Prayogshala Usage 30
13. Demonstration Video 30
Recording Link 31
https://drive.google.com/file/d/1l4DHBIS_2Sm9bUAmZvb_it4-PuaAsBy-/view?usp=sharing 31
14. Conclusion 31

GitHub Repository Link:

https://github.com/Neel2211/ACEest-Fitness-Gym

Recording Link

https://drive.google.com/file/d/1l4DHBIS_2Sm9bUAmZvb_it4-PuaAsBy-/view?usp=sharing

1. Executive Summary

This project implements a complete DevOps workflow for the ACEest Fitness & Gym application. The original ACEest application evolved through multiple incremental versions and was migrated to a Flask-based web application. Modern DevOps practices were applied including Git-based version control, automated unit testing using Pytest, Docker-based containerization, Continuous Integration using GitHub Actions, and build automation using Jenkins. The objective was to establish a reliable software delivery workflow that ensures code quality, repeatability, and deployment consistency.

---

2. Project Objective

The objective of this project is to:

- Develop a Flask-based fitness management application
- Implement version control using Git and GitHub
- Create a comprehensive Pytest test suite
- Containerize the application using Docker
- Implement CI/CD using GitHub Actions
- Configure Jenkins as a build validation environment
- Demonstrate incremental software evolution using branching and versioning

---

3. Application Overview

The ACEest Fitness & Gym application provides APIs for managing fitness-related activities.

Features Implemented

- Client Management
- Membership Tracking
- Fitness Program Generation
- Workout Management
- REST API Endpoints

Screenshot 1: Flask Application Running

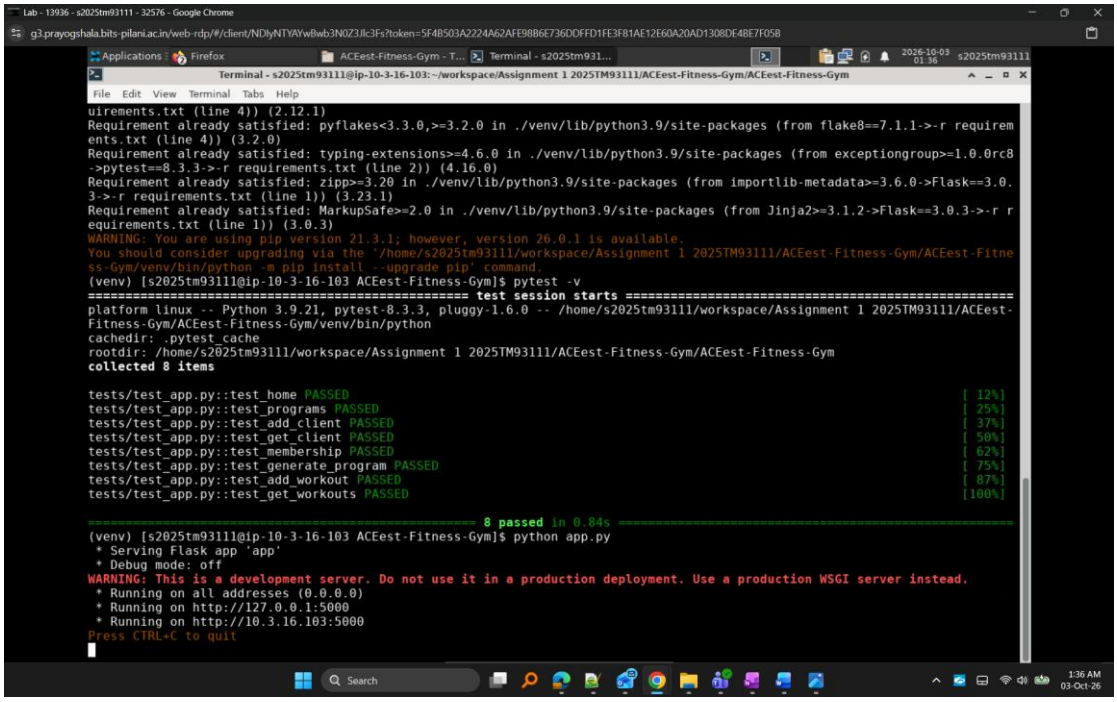


Figure 1. Flask application successfully running.

Screenshot 2: Application Response

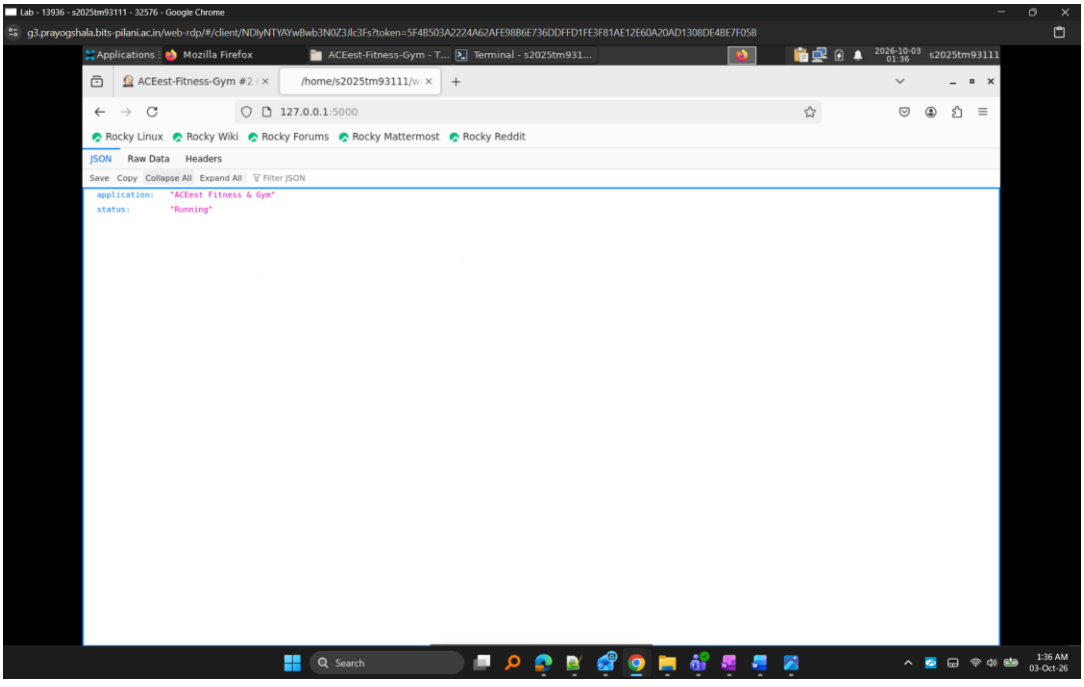


Figure 2. Flask application API response.

---

4. Version Control Strategy

To demonstrate version control maturity and software evolution, the supplied ACEest application versions were maintained using dedicated Git branches. Additional feature branches were used to isolate major DevOps enhancements. This approach follows industry Git workflows and enables controlled software evolution.

Version Evolution Table

Version	Description
v1.0	Initial gym management application
v1.1	Enhanced client management
v1.1.2	Improved fitness program management
v2.0.1	Workout tracking functionality
v2.1.2	Enhanced workout workflow
v2.2.1	Progress monitoring improvements
v2.2.4	Reporting and metrics enhancements
v3.0.1	Membership management
v3.1.2	Dashboard enhancements
v3.2.4	Final integrated desktop application

Feature Branches

- feature-flask
- feature-testing
- feature-docker
- feature-cicd
- feature-jenkins

Screenshot 3: Branches

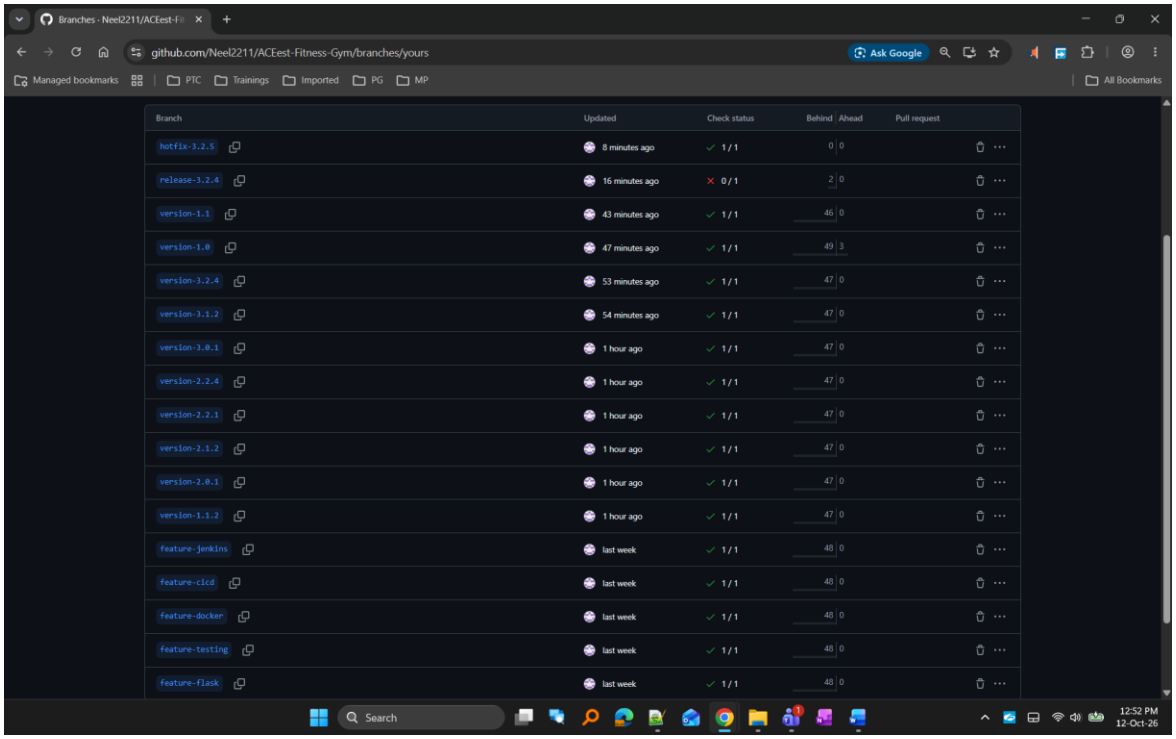


Figure 3. Version branches and feature branches maintained in GitHub.

Screenshot 4: Commit History

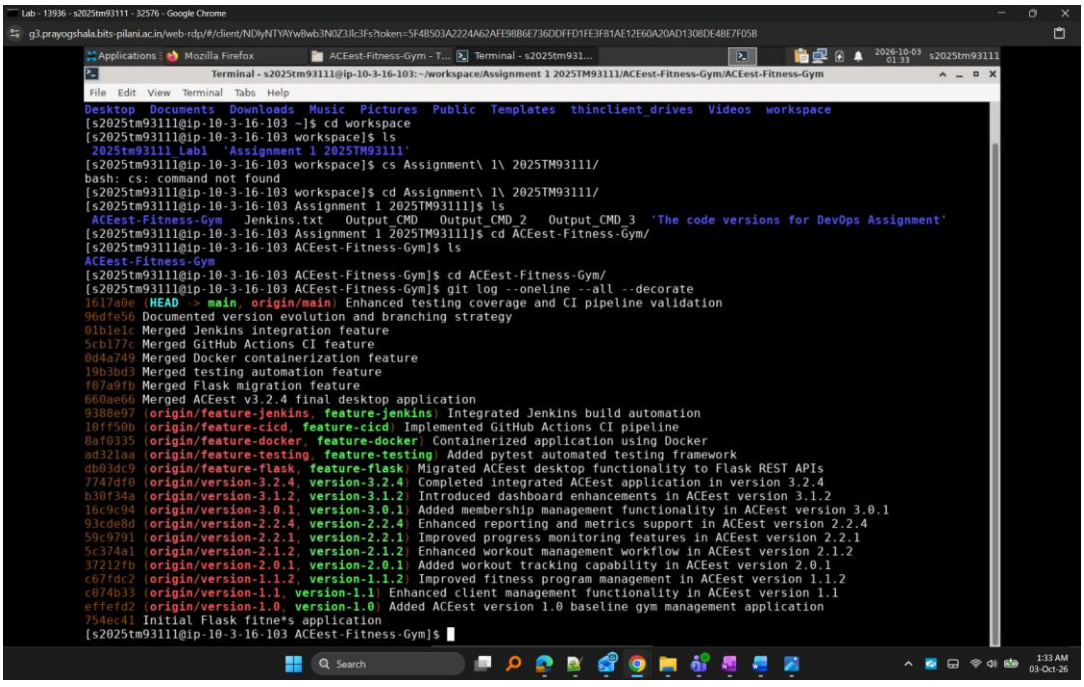


Figure 4. Meaningful commit history demonstrating incremental development

5. Unit Testing using Pytest

Pytest was integrated to perform automated validation of the Flask application before entering the build stage. Multiple test cases were developed to verify endpoint functionality and application behavior.

Test Coverage

- Home Endpoint
- Programs Endpoint
- Add Client
- Get Client
- Membership Endpoint
- Program Generation
- Add Workout
- Get Workouts

Screenshot 5: Pytest

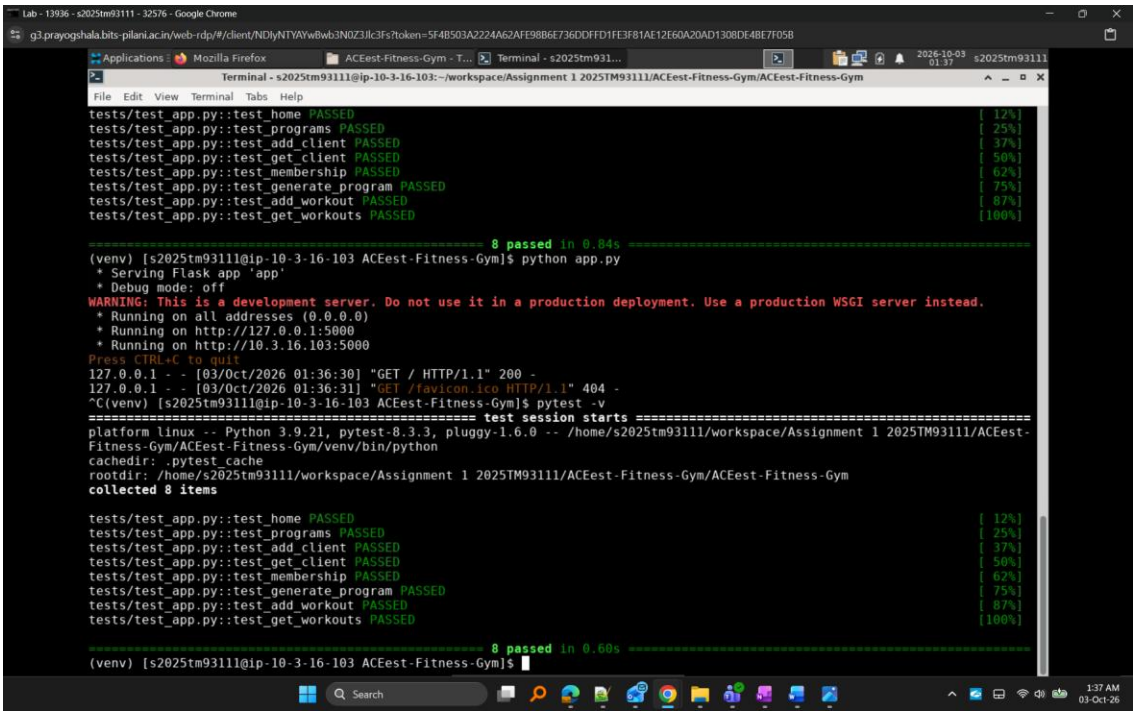


Figure 5. Pytest execution validating application functionality.

Test Execution Flow

Application Code

↓

Pytest Test Suite

↓

Endpoint Validation

↓

Pass / Fail Results

---

6. Docker Containerization

Docker was used to package the Flask application and its dependencies into a portable runtime environment. This eliminates dependency and environment inconsistencies and supports reproducible execution.

Screenshot 6: Docker Build

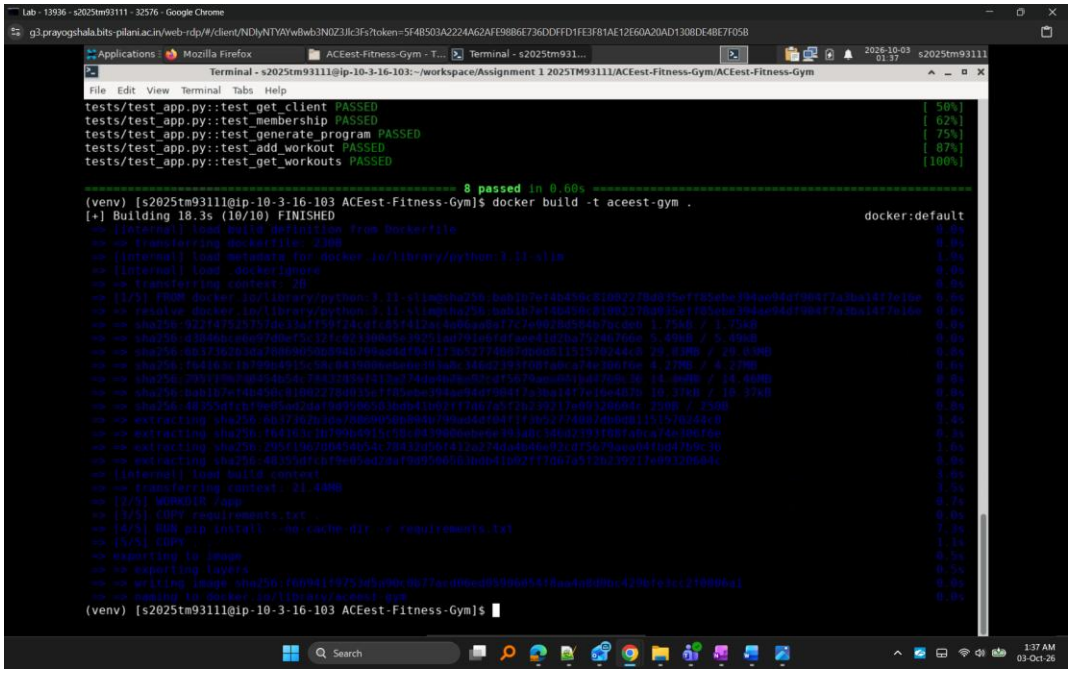


Figure 6. Docker image creation process.

Screenshot 7: Docker Run

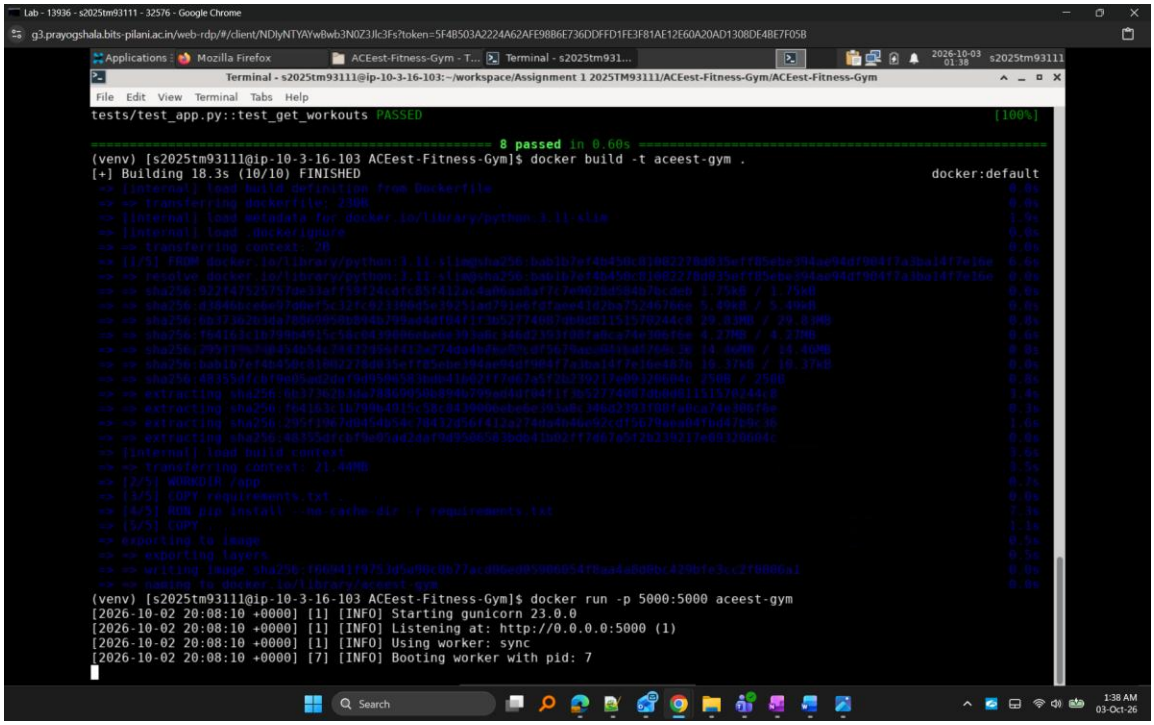


Figure 7. Application execution inside Docker container.

Screenshot 8: Application response from container

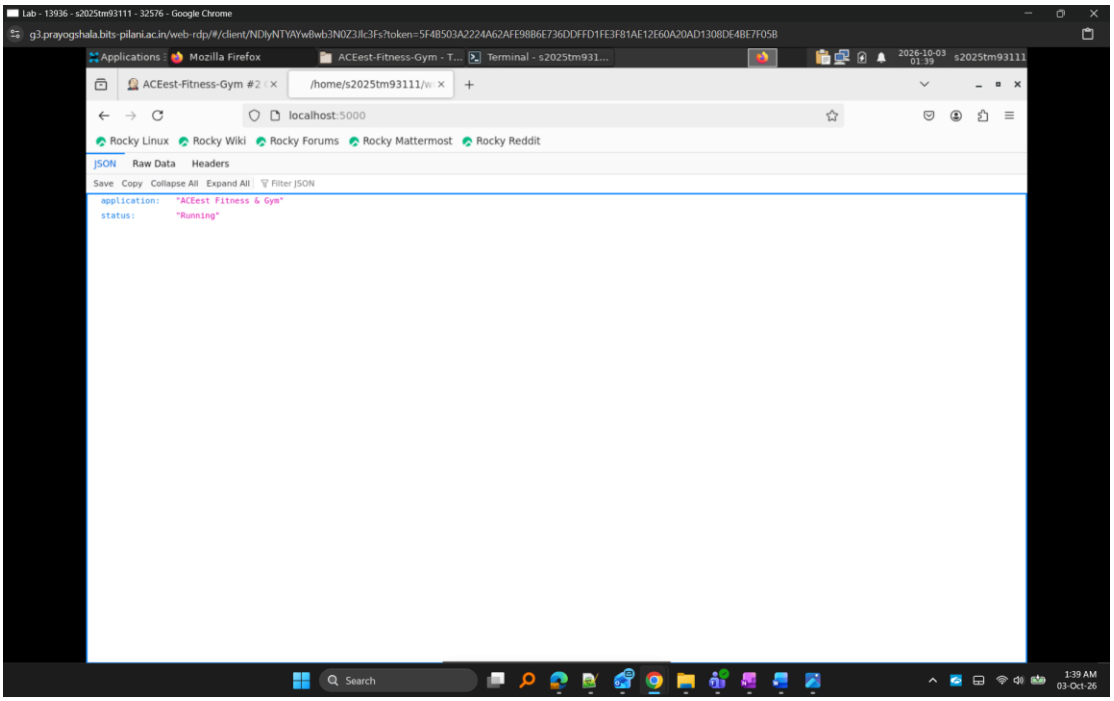


Figure 8. Flask application running inside Docker

7. GitHub Actions CI/CD Pipeline

GitHub Actions was configured to automatically execute continuous integration tasks whenever code is pushed to the repository.

The workflow performs:

Dependency Installation

↓

Lint Validation

↓

Pytest Execution

↓

Docker Image Creation

↓

Containerized Testing

CI/CD Flow Diagram

Developer Push

↓

GitHub Repository

↓

GitHub Actions Trigger

↓

Build & Lint

↓

Pytest Validation

↓

Docker Build

↓

Container Testing

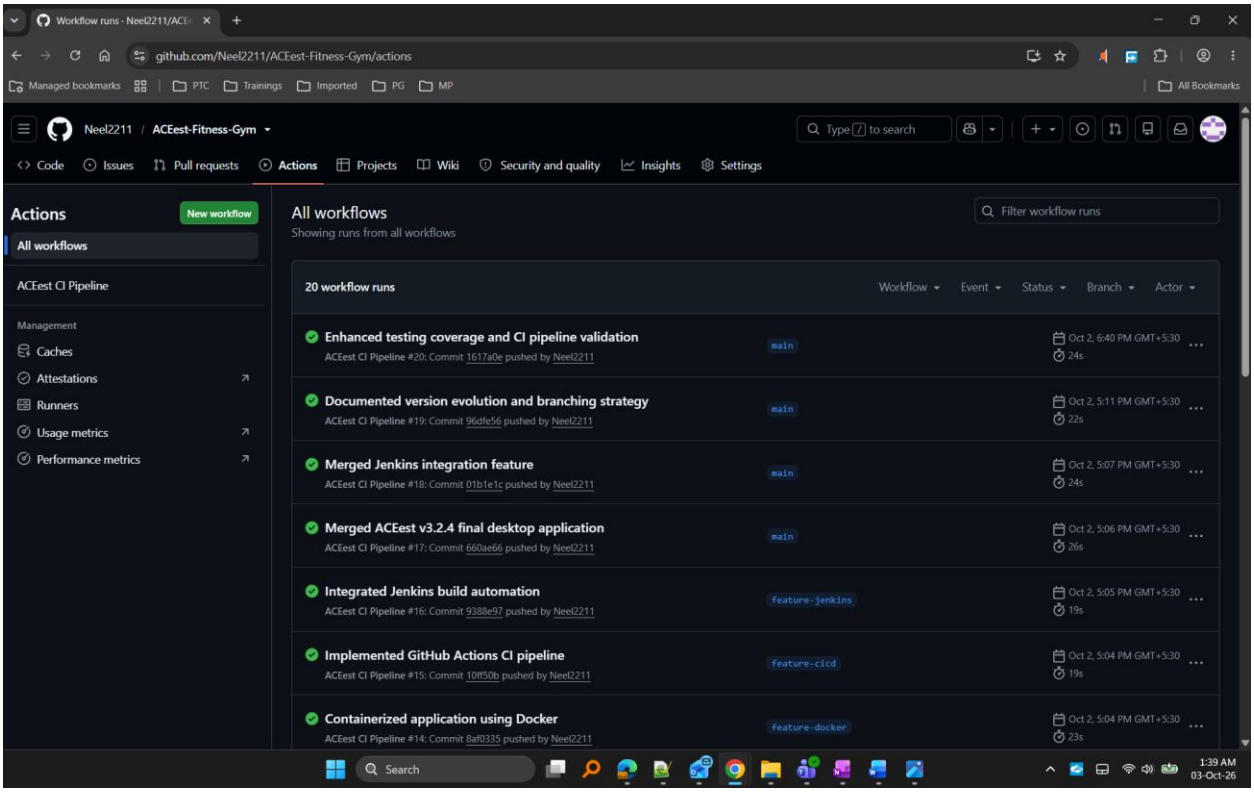
↓

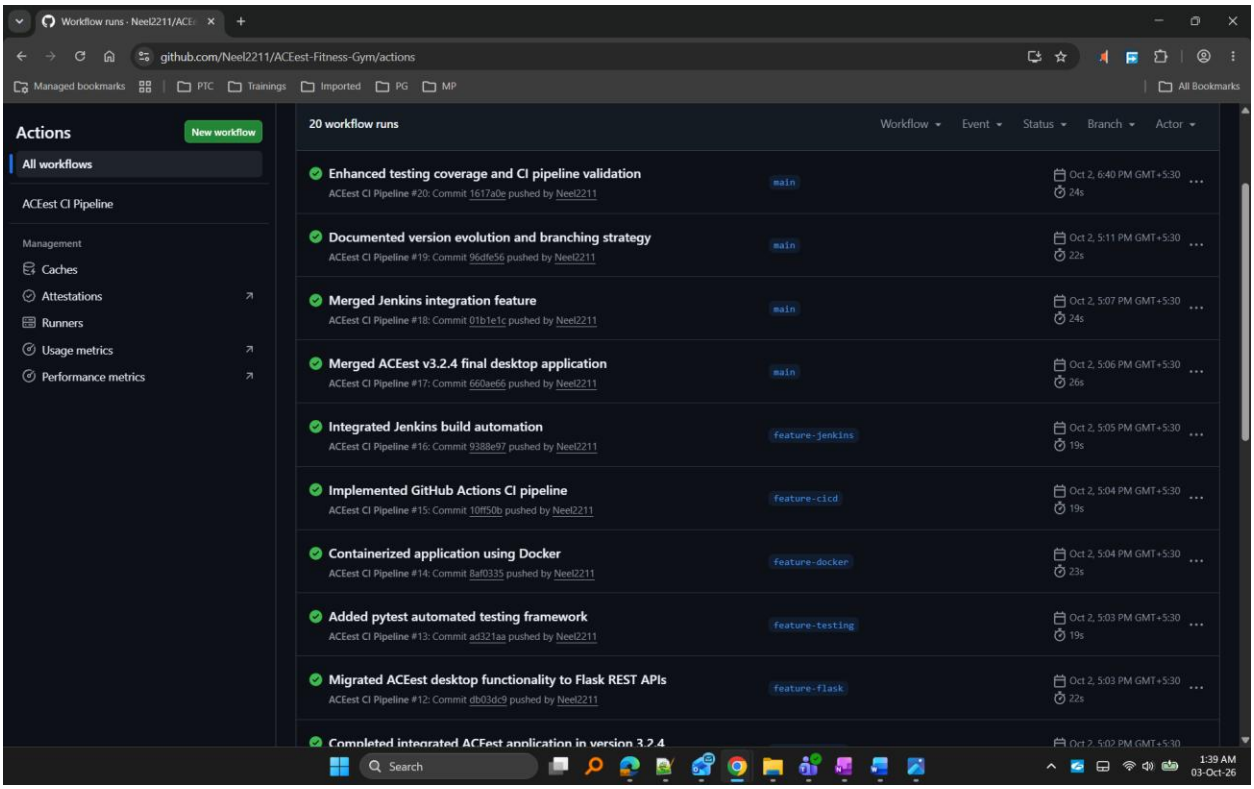
Successful Workflow

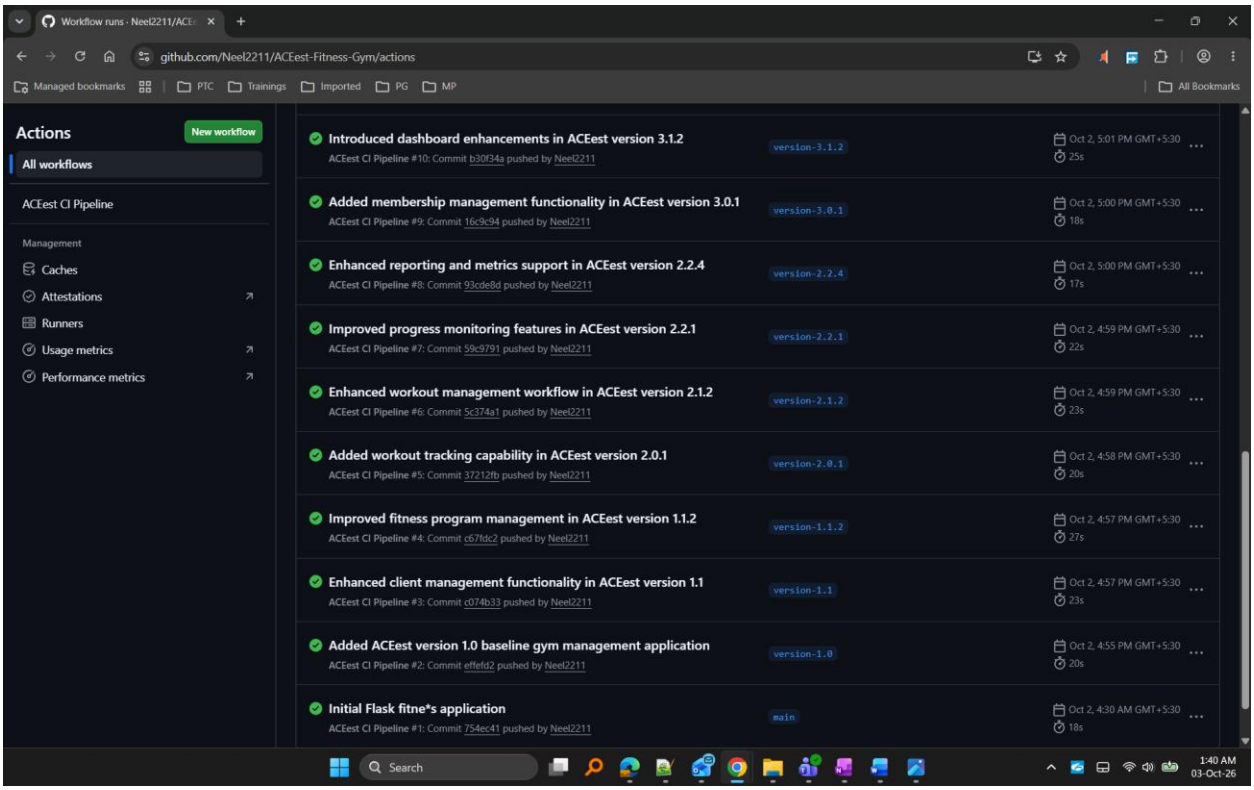
CI/CD Stages Executed

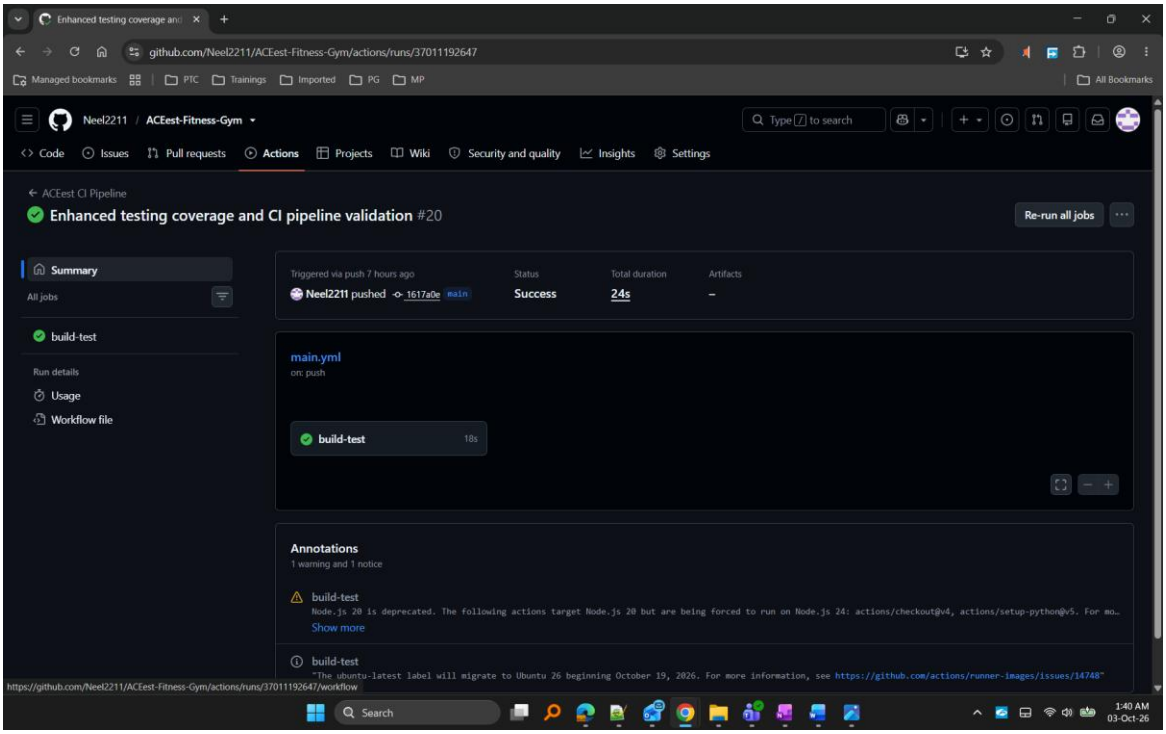
1. Source Code Checkout
2. Dependency Installation
3. Build & Lint Validation
4. Pytest Execution
5. Docker Image Build
6. Containerized Test Execution

Screenshot 9: GitHub Actions Overview




Workflow runs · Neel2211/ACEe
github.com/Neel2211/ACEest-Fitness-Gym/actions
Managed bookmarks
PTC
Trainings
Imported
PG
MP
All Bookmarks
Actions
New workflow
All workflows
ACEest CI Pipeline
Management
Caches
Attestations
Runners
Usage metrics
Performance metrics
20 workflow runs
Workflow
Event
Status
Branch
Actor
Enhanced testing coverage and CI pipeline validation
ACEest CI Pipeline #20: Commit 1617a0e pushed by Neel2211
main
Oct 2, 6:40 PM GMT+5:30
24s
Documented version evolution and branching strategy
ACEest CI Pipeline #19: Commit 96dfe56 pushed by Neel2211
main
Oct 2, 5:11 PM GMT+5:30
22s
Merged Jenkins integration feature
ACEest CI Pipeline #18: Commit 01b1e1c pushed by Neel2211
main
Oct 2, 5:07 PM GMT+5:30
24s
Merged ACEest v3.2.4 final desktop application
ACEest CI Pipeline #17: Commit 660ae66 pushed by Neel2211
main
Oct 2, 5:06 PM GMT+5:30
26s
Integrated Jenkins build automation
ACEest CI Pipeline #16: Commit 9388e97 pushed by Neel2211
feature-jenkins
Oct 2, 5:05 PM GMT+5:30
19s
Implemented GitHub Actions CI pipeline
ACEest CI Pipeline #15: Commit 10ff50b pushed by Neel2211
feature-cicd
Oct 2, 5:04 PM GMT+5:30
19s
Containerized application using Docker
ACEest CI Pipeline #14: Commit 8af0335 pushed by Neel2211
feature-docker
Oct 2, 5:04 PM GMT+5:30
23s
Added pytest automated testing framework
ACEest CI Pipeline #13: Commit ad321aa pushed by Neel2211
feature-testing
Oct 2, 5:03 PM GMT+5:30
19s
Migrated ACEest desktop functionality to Flask REST APIs
ACEest CI Pipeline #12: Commit db03dc9 pushed by Neel2211
feature-flask
Oct 2, 5:03 PM GMT+5:30
22s
Completed integrated ACEest application in version 3.2.4
Search
1:39 AM
03-Oct-26


Workflow runs · Neel2211/ACEe
github.com/Neel2211/ACEest-Fitness-Gym/actions
Managed bookmarks
PTC
Trainings
Imported
PG
MP
All Bookmarks
Actions
New workflow
All workflows
ACEest CI Pipeline
Management
Caches
Attestations
Runners
Usage metrics
Performance metrics
Introduced dashboard enhancements in ACEest version 3.1.2
ACEest CI Pipeline #10: Commit b30f34a pushed by Neel2211
version-3.1.2
Oct 2, 5:01 PM GMT+5:30
25s
Added membership management functionality in ACEest version 3.0.1
ACEest CI Pipeline #9: Commit 16c9c94 pushed by Neel2211
version-3.0.1
Oct 2, 5:00 PM GMT+5:30
18s
Enhanced reporting and metrics support in ACEest version 2.2.4
ACEest CI Pipeline #8: Commit 93cde8d pushed by Neel2211
version-2.2.4
Oct 2, 5:00 PM GMT+5:30
17s
Improved progress monitoring features in ACEest version 2.2.1
ACEest CI Pipeline #7: Commit 59c9791 pushed by Neel2211
version-2.2.1
Oct 2, 4:59 PM GMT+5:30
22s
Enhanced workout management workflow in ACEest version 2.1.2
ACEest CI Pipeline #6: Commit 5c374a1 pushed by Neel2211
version-2.1.2
Oct 2, 4:59 PM GMT+5:30
23s
Added workout tracking capability in ACEest version 2.0.1
ACEest CI Pipeline #5: Commit 37212fb pushed by Neel2211
version-2.0.1
Oct 2, 4:58 PM GMT+5:30
20s
Improved fitness program management in ACEest version 1.1.2
ACEest CI Pipeline #4: Commit c67fdc2 pushed by Neel2211
version-1.1.2
Oct 2, 4:57 PM GMT+5:30
27s
Enhanced client management functionality in ACEest version 1.1
ACEest CI Pipeline #3: Commit c074b33 pushed by Neel2211
version-1.1
Oct 2, 4:57 PM GMT+5:30
23s
Added ACEest version 1.0 baseline gym management application
ACEest CI Pipeline #2: Commit effefd2 pushed by Neel2211
version-1.0
Oct 2, 4:55 PM GMT+5:30
20s
Initial Flask fitne*s application
ACEest CI Pipeline #1: Commit 754ec41 pushed by Neel2211
main
Oct 2, 4:30 AM GMT+5:30
18s
Search
1:40 AM
03-Oct-26



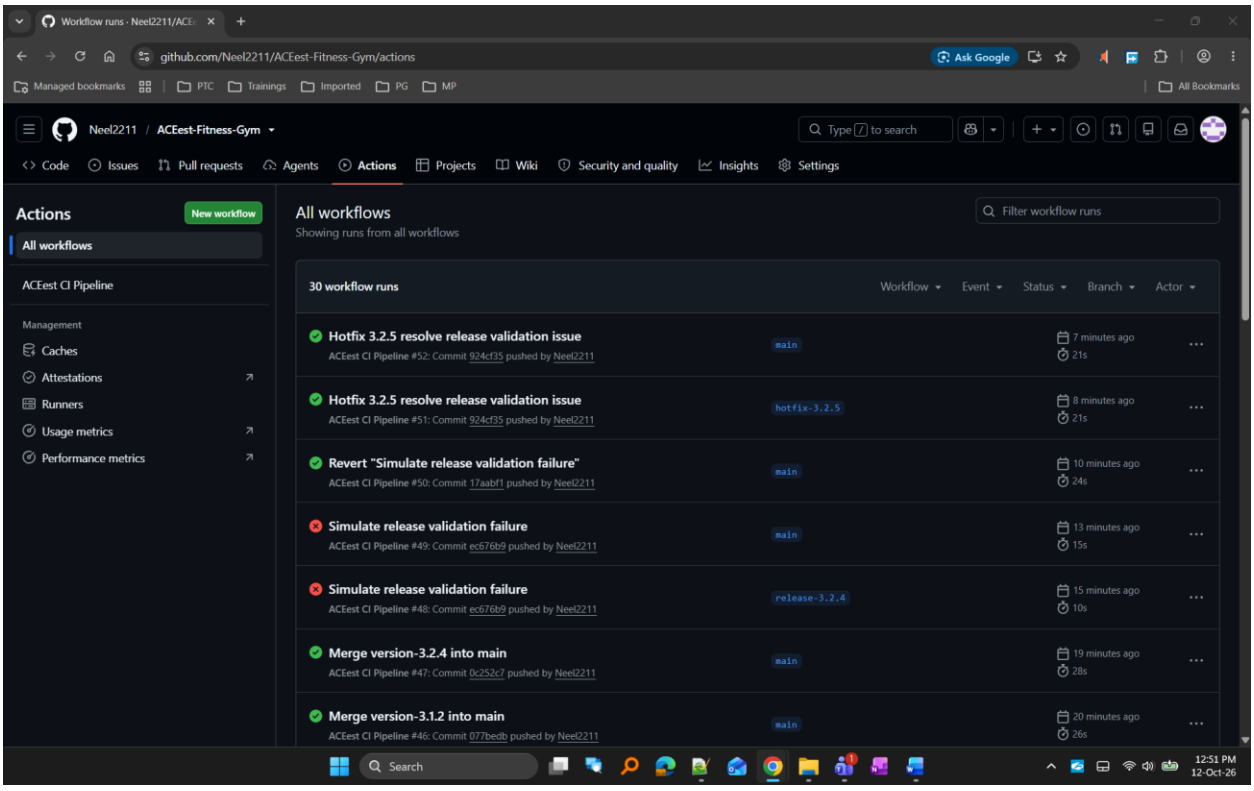


Figure 9. Successful GitHub Actions workflow runs

Screenshot 10: Workflow details

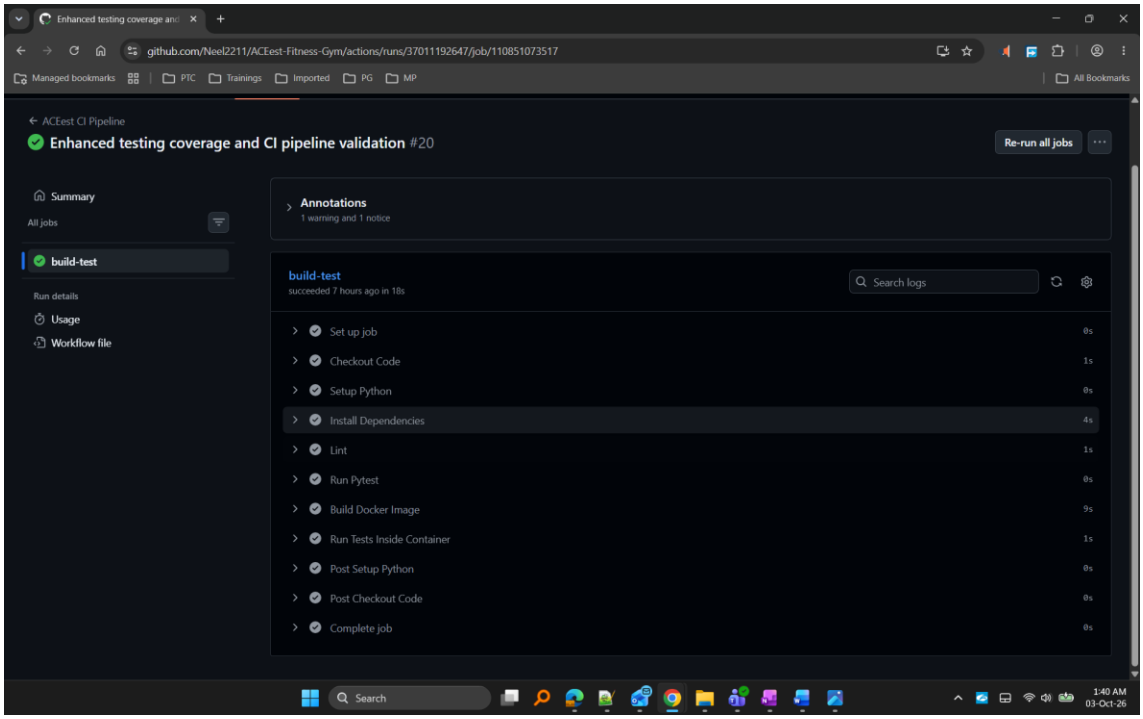


Figure 10. Build, Lint, Testing and Docker stages.

---

8. Jenkins Build & Quality Gate

Jenkins was configured as a secondary build validation environment. The pipeline automatically detects source code changes using SCM polling, retrieves the latest code from GitHub, validates the execution environment, installs project dependencies, and executes the automated Pytest suite. This provides a quality gate before release promotion and ensures that changes are validated in a controlled build environment.

Jenkins was configured with SCM polling to automatically trigger builds whenever changes were pushed to the monitored branch. This enabled continuous validation of version promotions, release candidates, rollback operations, and hotfix implementations.

Jenkins Workflow

GitHub Repository

↓

SCM Polling

↓

Automatic Jenkins Trigger

↓

Environment Validation

↓

Install Dependencies

↓

Run Pytest

↓

Build Validation

↓

SUCCESS / FAILURE

Screenshot 11: Jenkins Pipeline Configuration

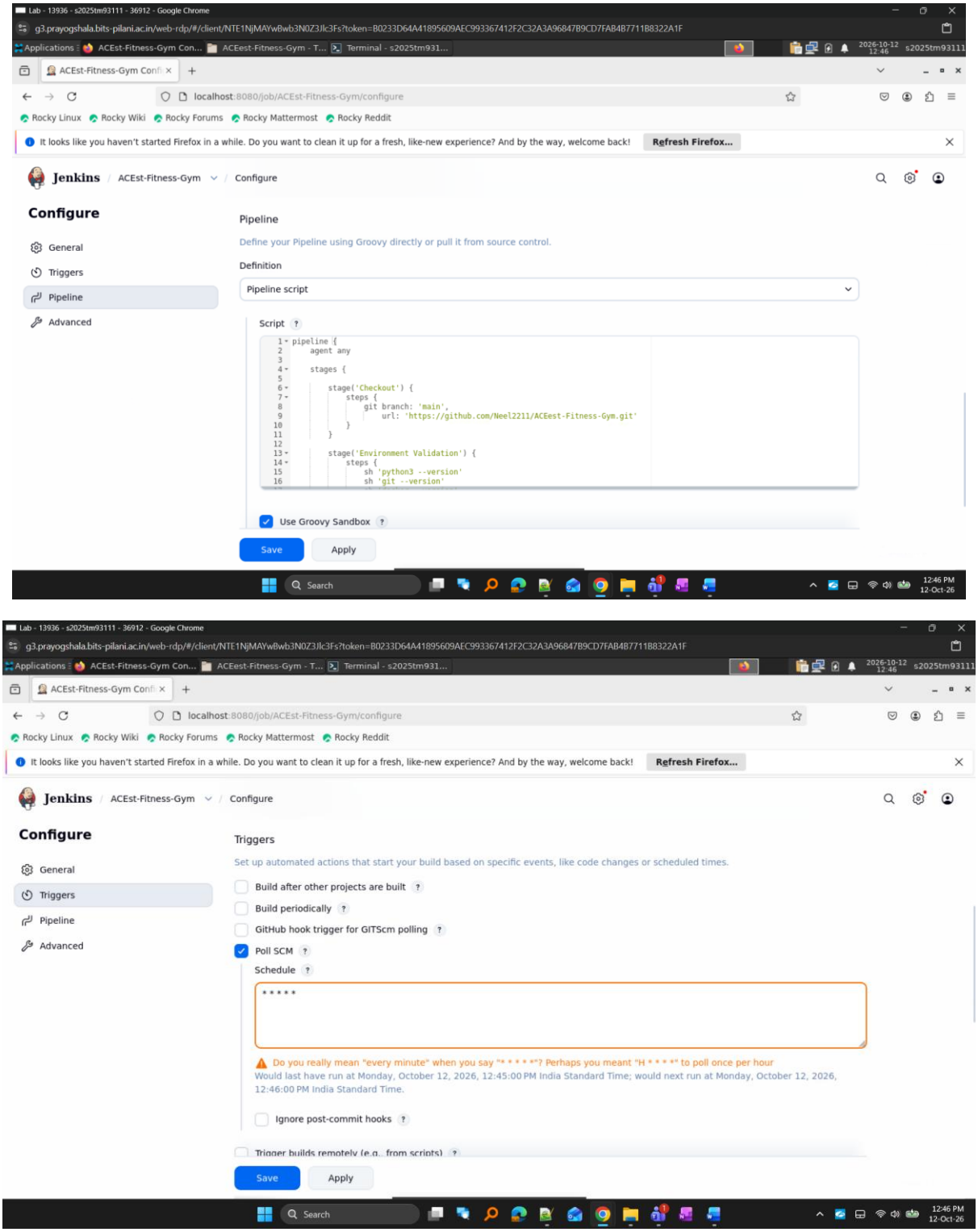


Figure 11. Jenkins pipeline configuration used for build validation, dependency installation, and automated testing.

Screenshot 12: Automatic Jenkins Trigger: Build started by SCM change

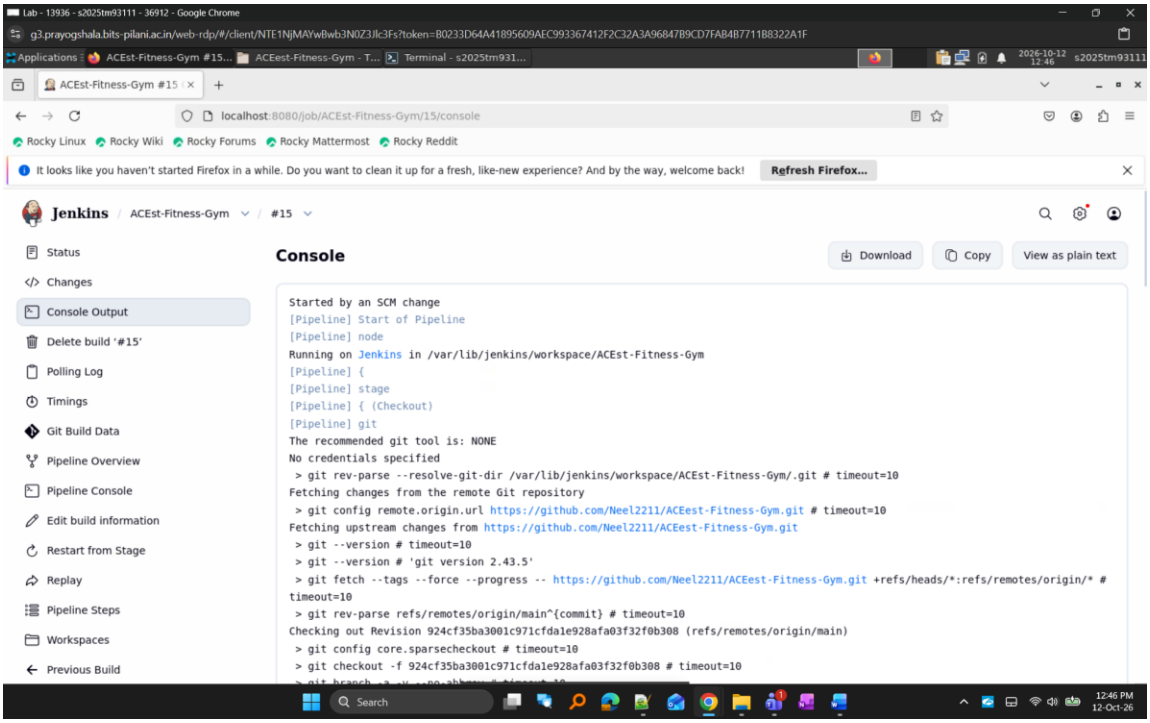


Figure 12. Jenkins automatically triggered following a source code push.

Screenshot 13: Environment Validation

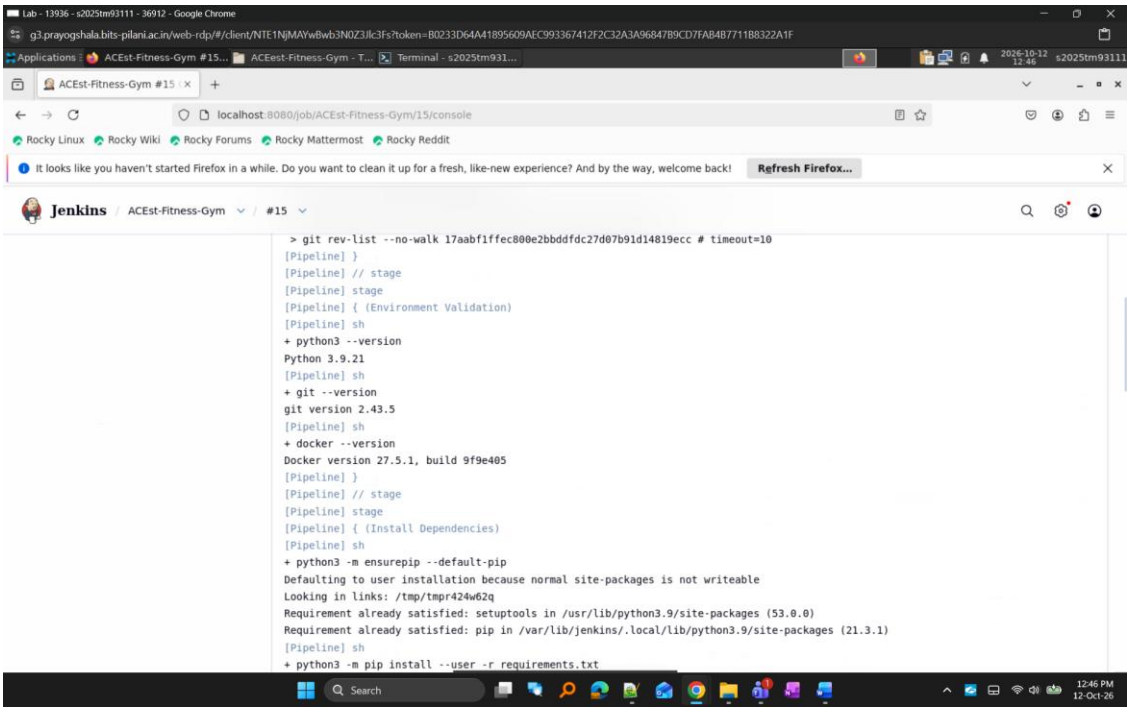


Figure 13. Jenkins environment validation confirming Python, Git, and Docker availability.

Screenshot 14: Dependency Installation

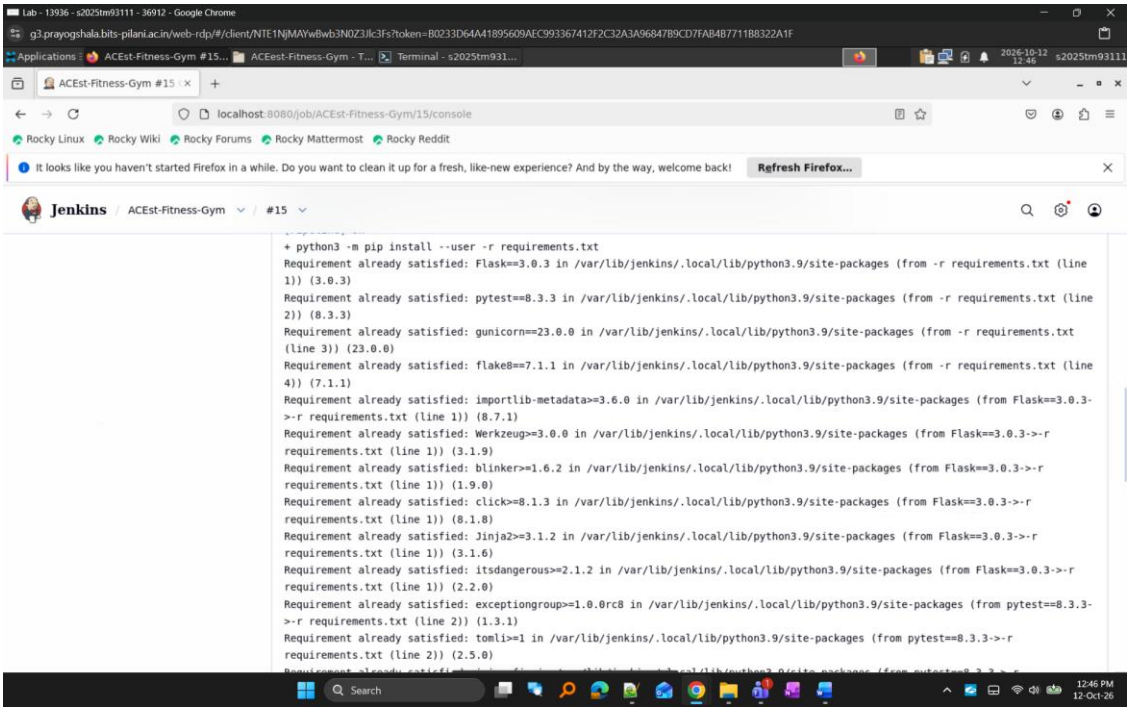


Figure 14. Jenkins building the application environment and installing project dependencies

Screenshot 15: Pytest Execution

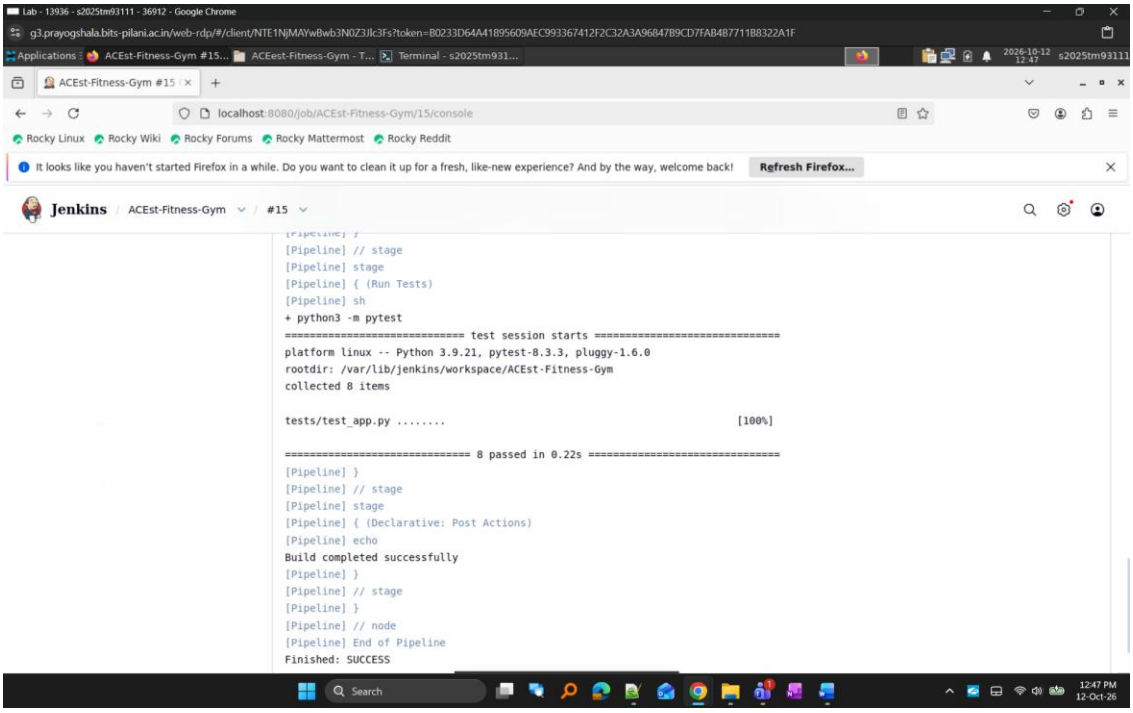


Figure 15. Jenkins executing automated Pytest validation with all test cases passing successfully

Screenshot 16: Build Success

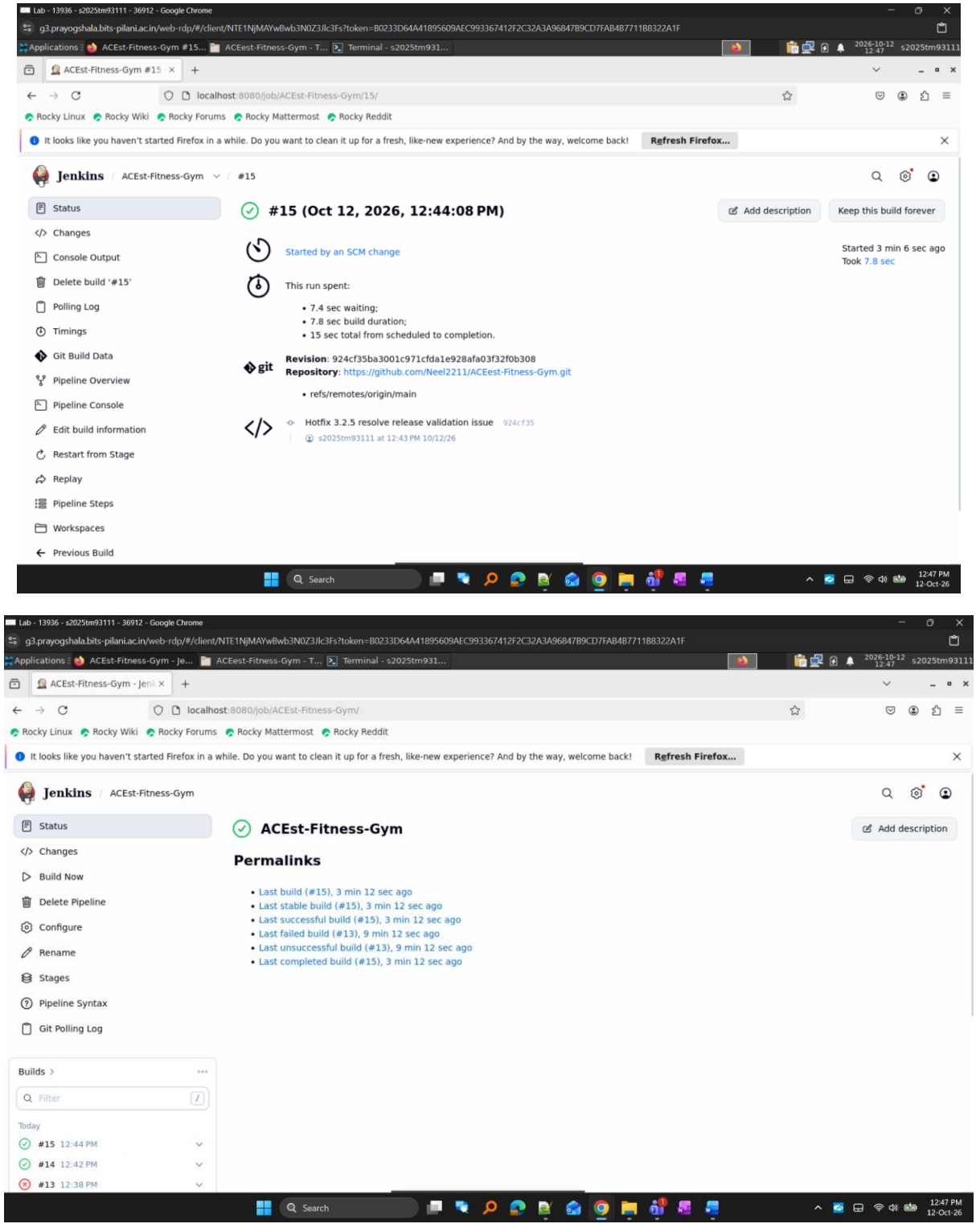


Figure 16. Successful Jenkins BUILD execution and quality gate validation

9. Release Management, Rollback and Hotfix Strategy

As part of the DevOps workflow, multiple ACEest release versions were incrementally promoted and validated through Jenkins. Each version was merged into the main branch and automatically validated through SCM-triggered Jenkins builds. This simulated a real-world release management workflow.

A release validation failure was intentionally introduced in the release branch to demonstrate Jenkins build validation, rollback strategy, and hotfix implementation.

Release Validation Workflow

Version-1.0

↓

Automatic Jenkins Build

↓

SUCCESS

Version-1.1

↓

Automatic Jenkins Build

↓

SUCCESS

Version-1.1.2

↓

Automatic Jenkins Build

↓

SUCCESS

...

Version-3.2.4

↓

Automatic Jenkins Build

↓

SUCCESS

Release-3.2.4

↓

Validation Failure

↓

FAILED BUILD

Rollback

↓

Automatic Jenkins Build

↓

SUCCESS

Hotfix-3.2.5

↓

Automatic Jenkins Build

↓

SUCCESS

Screenshot 17: Incremental Version Builds

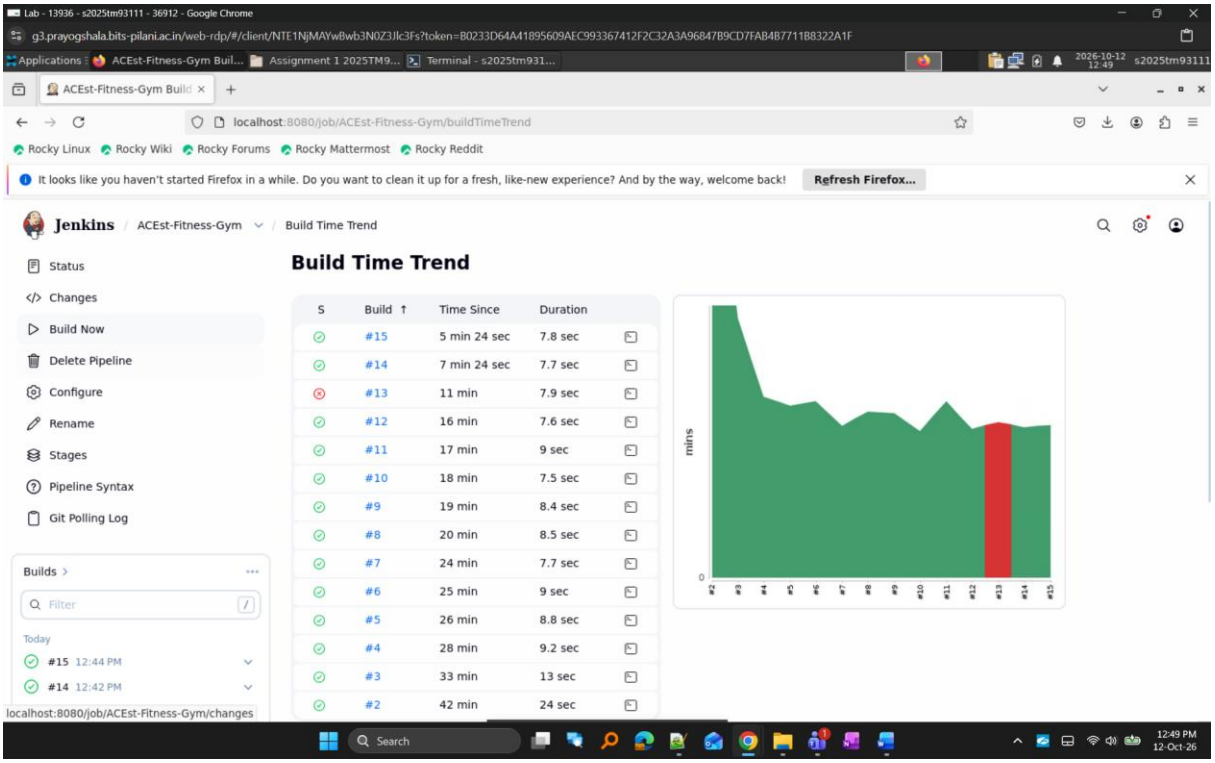
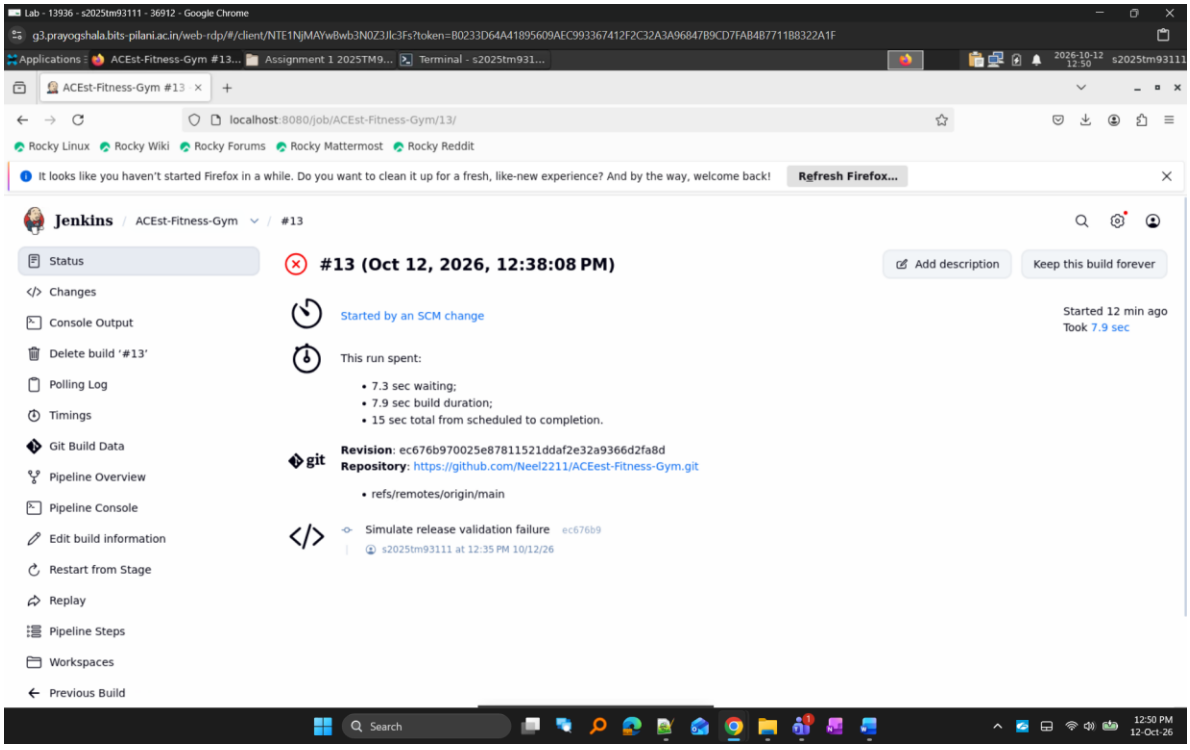


Figure 17. Multiple version releases validated through automatic Jenkins builds.

Screenshot 18: Release Failure



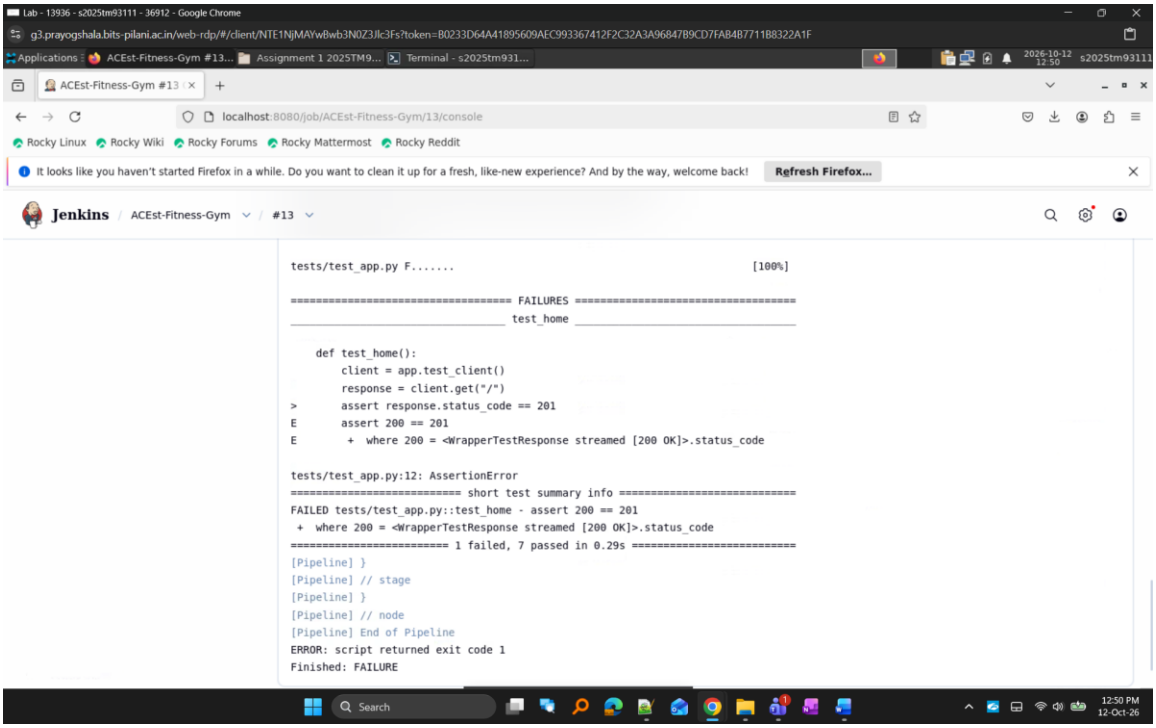
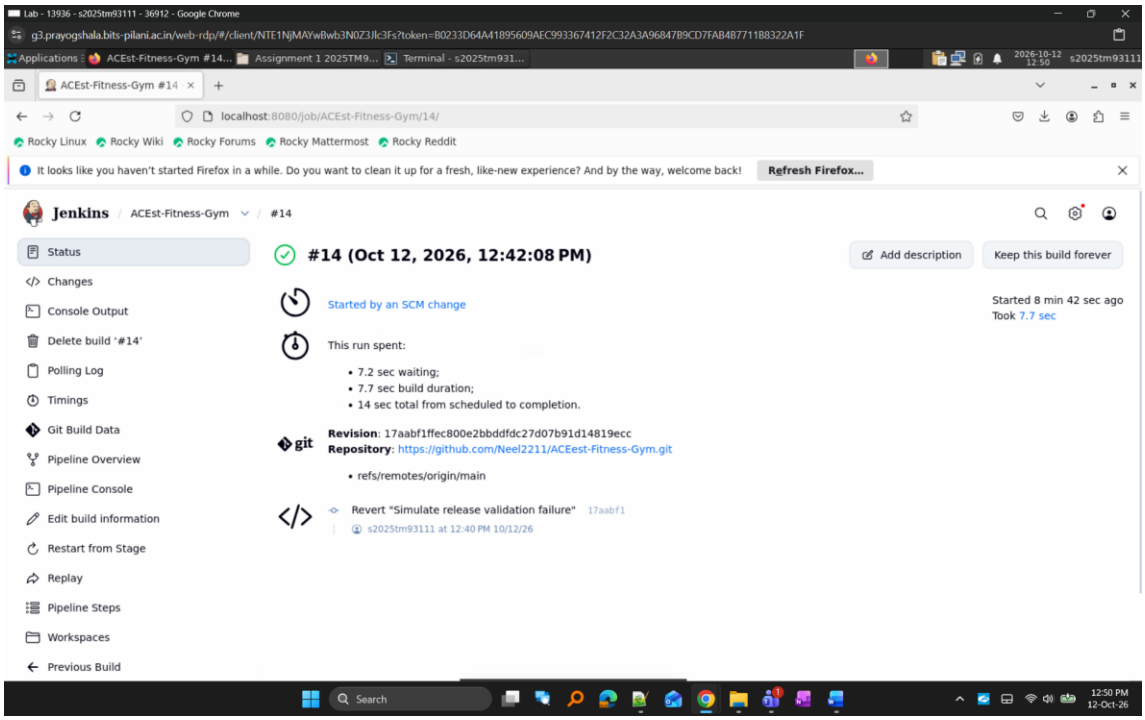


Figure 18. Jenkins detecting validation failure in release-3.2.4.

Jenkins successfully detected the release validation issue and prevented the faulty release from progressing further in the pipeline.

Screenshot 19: Rollback Success



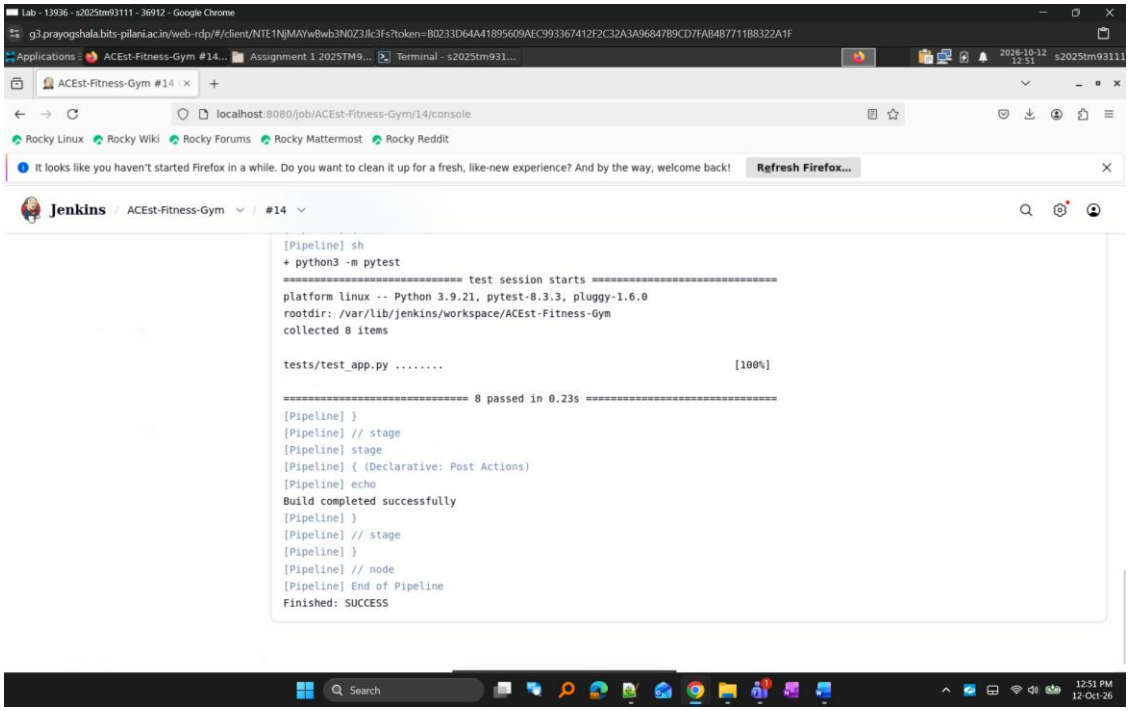


Figure 19. Successful rollback to a stable release version.

The rollback operation restored the application to the last known stable release and the Jenkins build completed successfully.

Screenshot 20: Hotfix Success

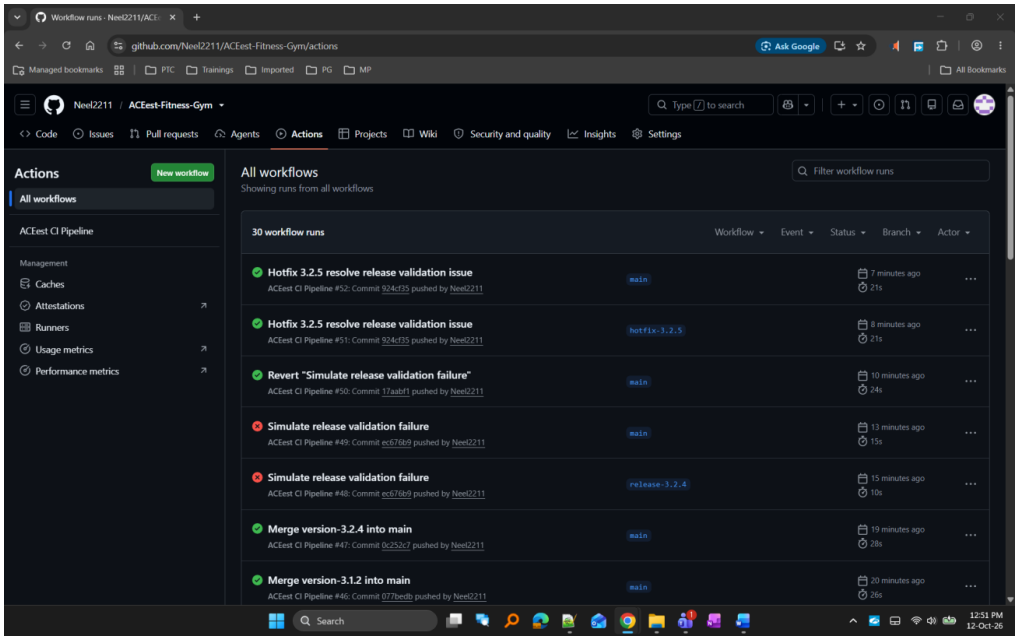


Figure 20. Successful validation of hotfix-3.2.5 through the automated CI/CD pipeline

10. Repository Structure

```
ACEest-Fitness-Gym/
│
├── app.py
├── requirements.txt
├── Dockerfile
├── Jenkinsfile
├── README.md
│
├── tests/
│   └── test_app.py
│
├── legacy_versions/
│   ├── Aceestver-1.0.py
│   ├── ...
│   └── Aceestver-3.2.4.py
│
└── .github/
    └── workflows/
        └── main.yml
```

Screenshot 21: Repository Structure

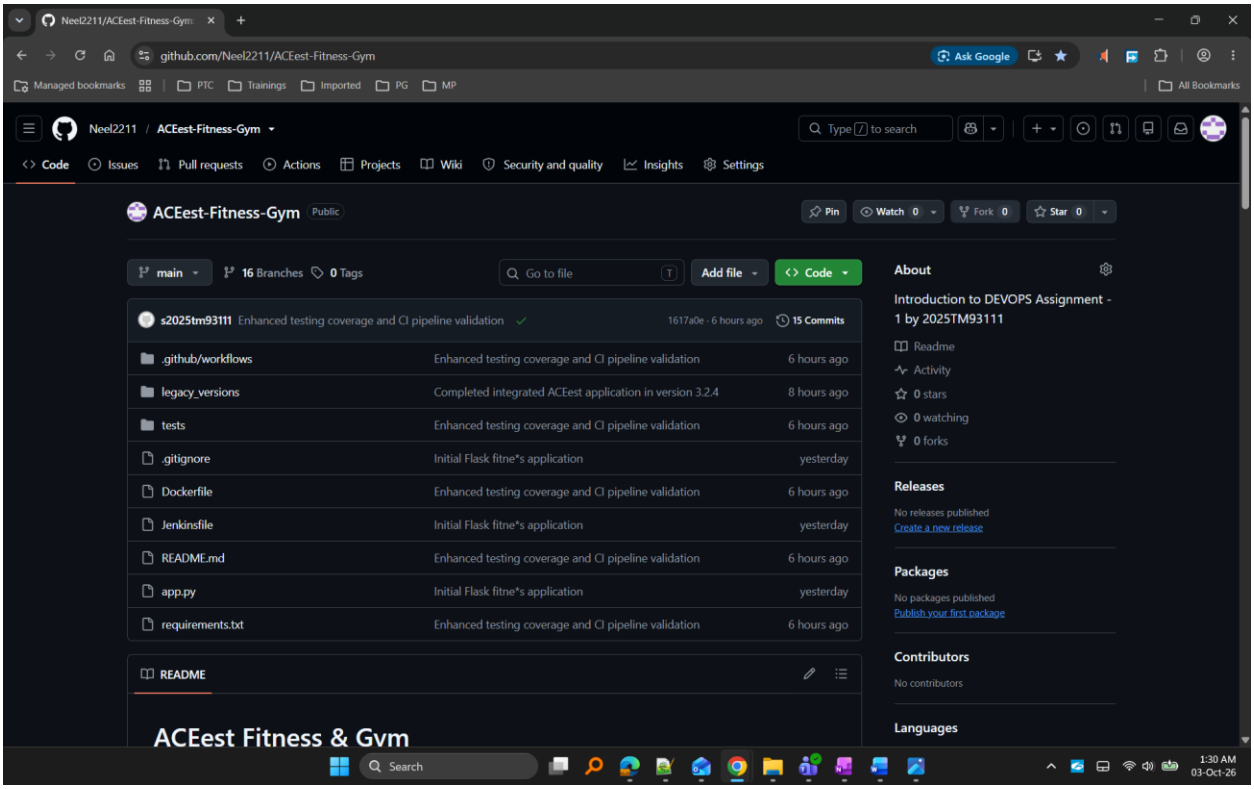


Figure 21. Project repository structure

GitHub Repository: https://github.com/Neel2211/ACEest-Fitness-Gym

---

11. Project Documentation (README.md)

Description

A professional README.md file was created to support project onboarding and maintenance. The document provides setup instructions, testing procedures, Docker execution steps, CI/CD workflow details, Jenkins integration information, repository structure, and version evolution details. This ensures that other developers can easily understand, build, test, and maintain the project.

README Contents

The README includes:

- Project Overview

- Technology Stack
- Repository Structure
- Local Setup Instructions
- Application Execution Instructions
- Pytest Execution Steps
- Docker Build and Run Commands
- GitHub Actions Workflow Overview
- Jenkins Integration Overview
- Version Evolution Strategy
- API Endpoints

Screenshot 22: README Overview

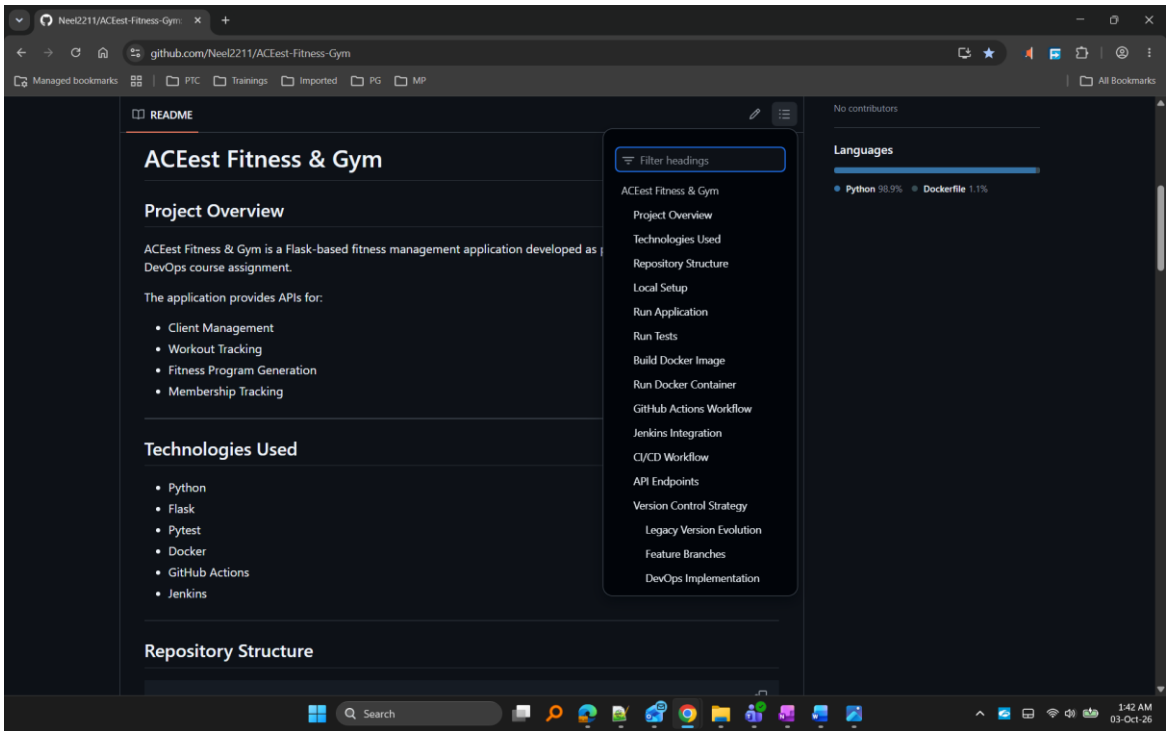


Figure 22. Project README documentation

Screenshot 23: README CI/CD Workflow

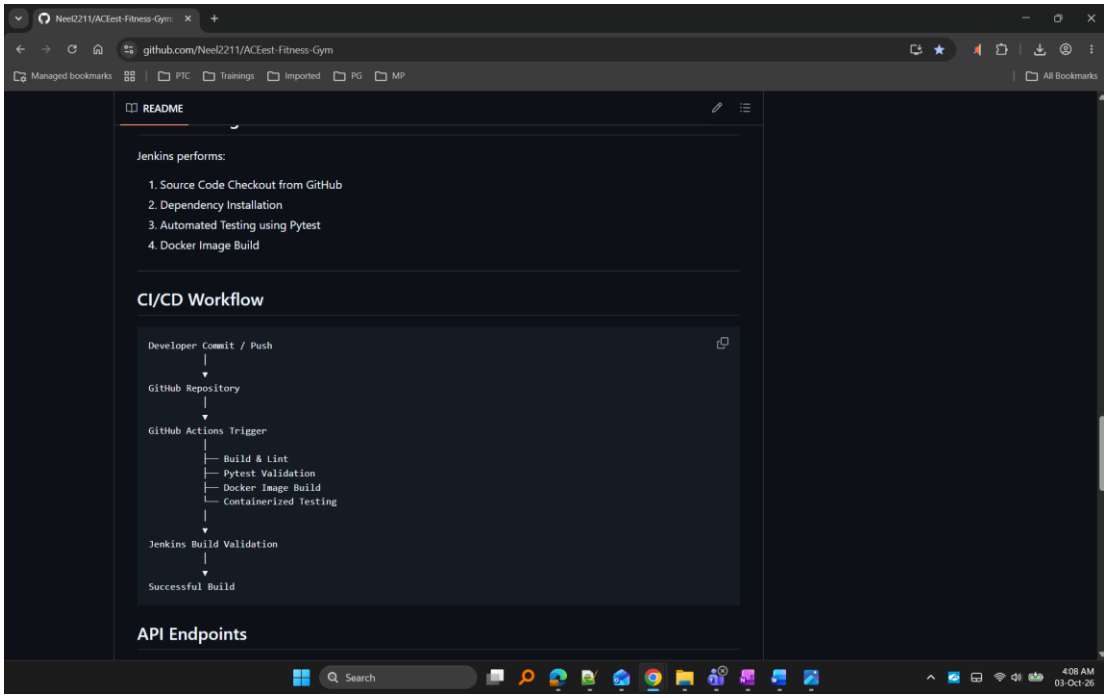


Figure 23. CI/CD workflow documented in README

Screenshot 24: README Setup Instructions

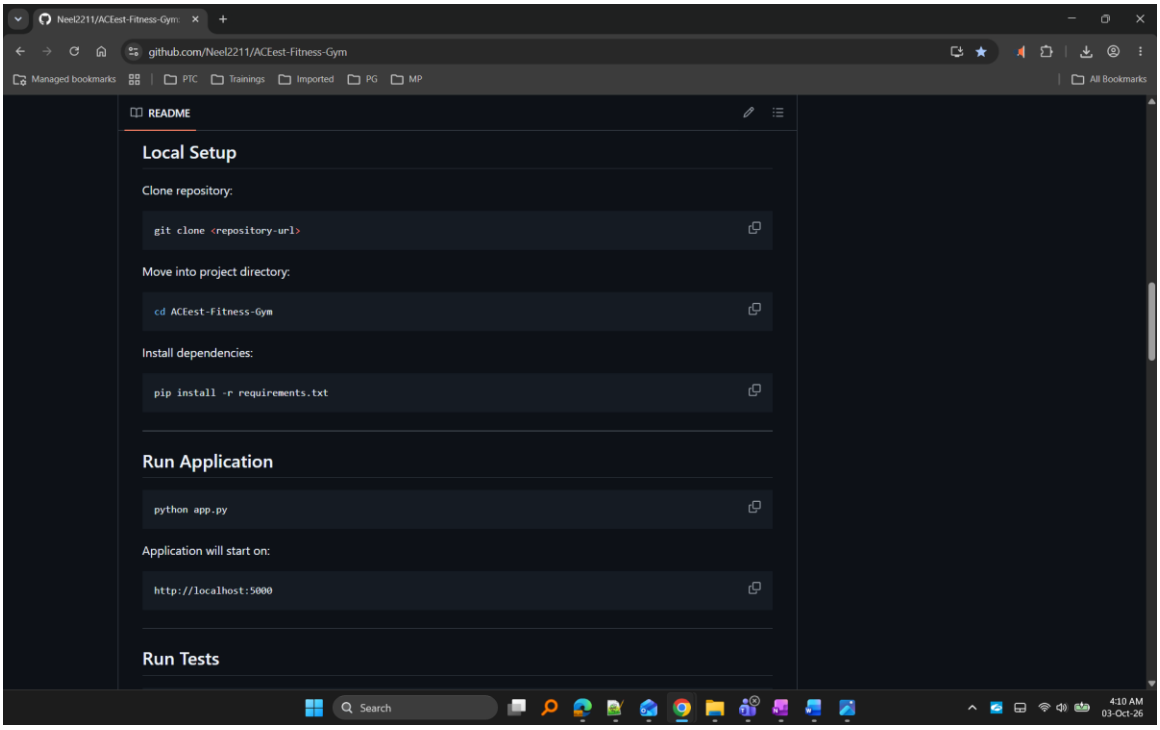


Figure 24.Installation, execution, and testing instructions documented in README.

12. Prayogshala Usage

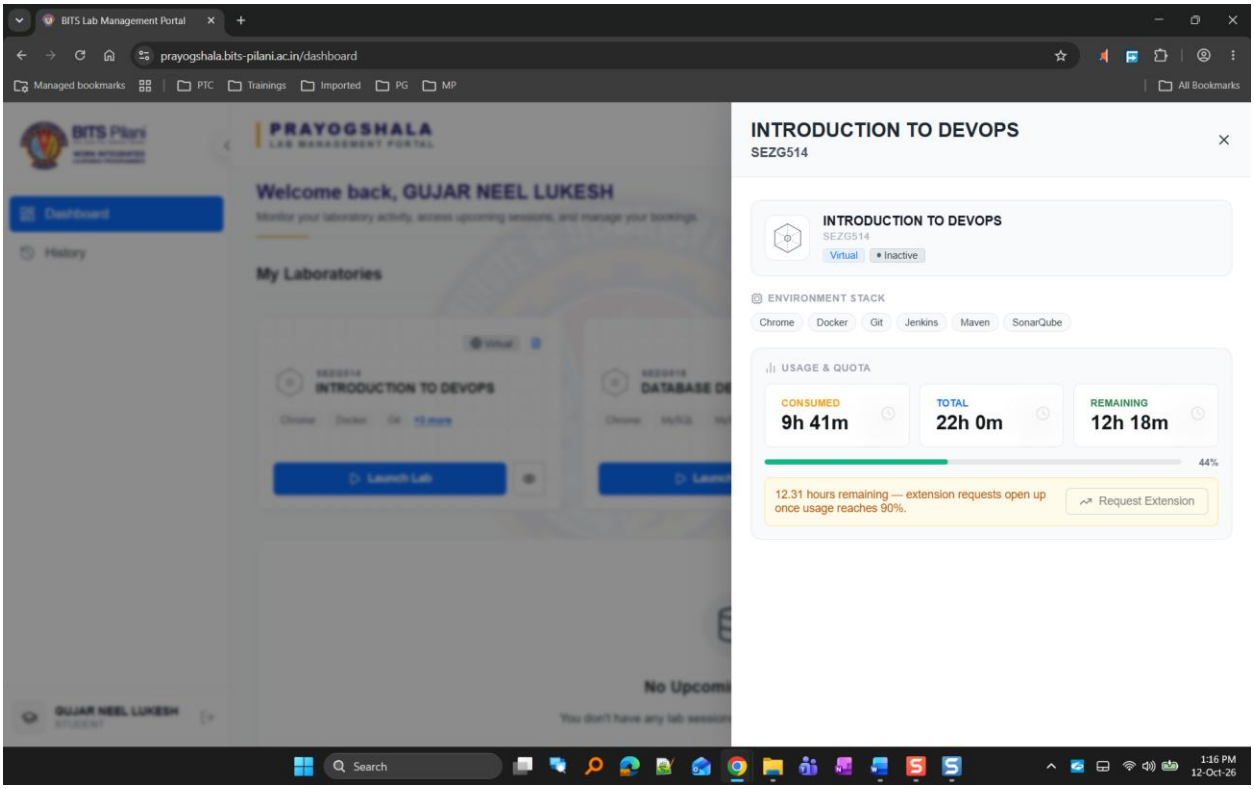


Figure 25. Prayogshala Usage

---

13. Demonstration Video

A demonstration video was recorded to showcase:

- Git version control strategy
- Incremental version builds
- Jenkins automatic build triggering using SCM polling
- GitHub Actions workflow execution
- Release failure scenario
- Rollback strategy
- Hotfix implementation

Recording Link

https://drive.google.com/file/d/1l4DHBIS_2Sm9bUAmZvb_it4-PuaAsBy-/view?usp=sharing

---

14. Conclusion

The ACEest Fitness & Gym project successfully demonstrates an end-to-end DevOps workflow including application development, version control, automated testing, Docker containerization, CI/CD automation, Jenkins-based build verification, release validation, rollback, and hotfix strategies. Multiple version releases were incrementally validated through SCM-triggered Jenkins builds, while failed-release, rollback, and hotfix workflows demonstrated practical release management concepts used in modern DevOps environments.
